While loops

CSC103 Programming Fundamentals / Lecture 10

Dr. Muhammad Farhan

Associate Professor of Computer Science
Department of Computer Science
COMSATS University Islamabad
Sahiwal Campus

Fall 2026

The problem for this lecture

Design a sentinel loop for marks ending at -1. Print the count and average, or No data when the sentinel arrives first.

Initialise sum and count to zero, then read the first mark.

Learning objectives

- Design terminating while loops
- Use counter, sentinel and flag control
- Trace state across iterations

CLO-2
The while structure
Counter-controlled repetition
Sentinel-controlled repetition
Flag-controlled repetition

The while structure

- A while loop checks its condition before each iteration.
- Its body may run zero times.
- Initialisation, condition and progress must agree.

Example

```
int i = 1;
while (i <= 3) {
  System.out.println(i);
  i++;
}
```

Counter-controlled repetition

- Use a counter when the number of repetitions is known.
- An accumulator starts at an appropriate identity, such as zero for a sum.
- State the range clearly.

Example

```
int total = 0;
int i = 1;
while (i <= 4) {
  total += i;
  i++;
}
```

Sentinel-controlled repetition

- A sentinel marks the end of input and is not ordinary data.
- Read before the loop and again after processing each accepted value.
- Do not include the sentinel in a sum or count.

Example

```
int x = in.nextInt();
while (x != -1) {
  total += x;
  count++;
  x = in.nextInt();
}
```

Flag-controlled repetition

- A Boolean flag can record whether a condition has been achieved.
- Every path must permit progress toward stopping.
- A loop that never changes its condition may not terminate.

Example

```
boolean found = false;
int candidate = 1;
while (!found && candidate <= 10) {
  found = candidate % 7 == 0;
  candidate++;
}
```

A loop maintains a useful relationship

- After processing k marks, count should equal k and sum should contain their total.
- The next iteration extends this relationship with one accepted mark.
- This makes the final calculation easier to justify.

Example

```
Before input: count=0, sum=0
After 60: count=1, sum=60
After 80: count=2, sum=140
```

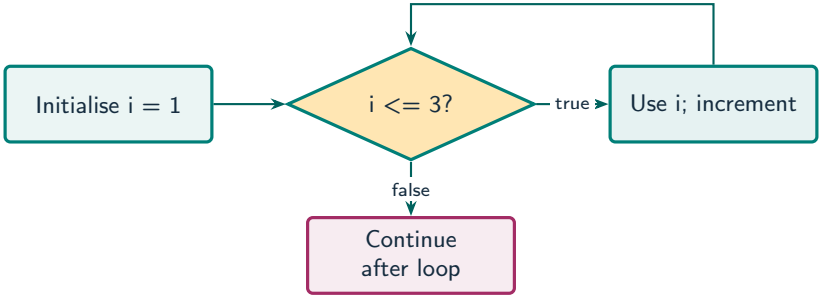
The sentinel is control data

- The stop value ends repetition and contributes neither a mark nor a count.
- Checking before accumulation prevents a biased sum.
- An immediate sentinel leaves count at zero, so there is no average to divide for.

Example

```
Input: 60 80 -1
Data values: 60 and 80
Sentinel: -1
```

A while loop checks before every iteration



What to notice

The back edge returns to the condition. A false first check gives zero iterations.

Key distinctions

Control style	Stop rule	Typical use
Counter	Required number processed	Read exactly 5 marks
Sentinel	Special input encountered	Read until -1
Flag	State becomes true or false	Stop after a match

These distinctions determine which operation or control structure fits the problem.

First example: execution

```
int i = 1;
int total = 0;
while (i <= 4) {
  total += i;
  i++;
}
System.out.println(total);
```

First example: result

10
After bodies: (i,total) = (2,1), (3,3), (4,6), (5,10).
The condition fails when i is 5.

Sentinel-controlled repetition
Flag-controlled repetition

Guided practice

Design a sentinel loop for marks ending at -1. Print the count and average, or No data when the sentinel arrives first.

Attempt the solution before continuing.
Use the definitions and examples from this lecture.

Solution strategy

- Initialise sum and count to zero, then read the first mark.
- While the mark is not -1, accumulate it and read the next mark.
- If count is positive, divide using floating-point arithmetic. Otherwise print No data.

Sample console input
60 80 -1

Complete solution (1/2)

```
public class Solution10 {
  public static void main(String[] args) throws Exception {
    java.util.Scanner in = new java.util.Scanner(System.in);
    int sum = 0;
    int count = 0;
    int mark = in.nextInt();
    while (mark != -1) {
      sum += mark;
```

Complete solution (2/2)

```
      count++;
      mark = in.nextInt();
    }
    if (count == 0) System.out.println("No data");
    else System.out.println((double) sum / count);
  }
}
```

Execution trace

Input	mark != -1	sum after body	count after body
60	true	60	1
80	true	140	2
-1	false	140 unchanged	2 unchanged

Follow the changing state in execution order.

Solution result

70.0

Why this result follows
If count is positive, divide using floating-point arithmetic. Otherwise print No data.

A common incorrect approach

```
int i = 1;
while (i <= 5) { System.out.println(i); }
```

Example

`i never changes. Add i++ inside the loop so the condition eventually becomes false.`

Boundary and failure checks

Check the stated input assumptions.
Use while to compute the digit sum of a nonnegative integer. Test 0, 7 and 204.

Initialise sum=0 and count=0. Add only non-sentinel marks. Divide (double) sum by count only when count > 0. Inputs 60,80,-1 give count 2 and average 70.

Check your understanding

Can while run zero times? Why must a sentinel be outside the normal input domain?

Write a prediction and explain the rule that supports it.

Answer and explanation

Yes, if the initial condition is false. Otherwise the program cannot distinguish a data value from the stop signal.

i never changes. Add i++ inside the loop so the condition eventually becomes false.

Compare cases and predict outcomes

Check	i	Action
First	1	Use 1; set i to 2
Third	3	Use 3; set i to 4
Final	4	Exit without using 4

Discuss

Explain each expected result before running the example. Which case would reveal a wrong assumption?

Apply the idea

Independent practice

Write a while loop that adds the odd integers 1, 3, 5 and 7. State the final counter value and the number of condition checks.

1. Work through the input by hand.
2. Write or correct the program.
3. Justify the expected result.

Solution and reasoning

```
int i = 1;
int sum = 0;
while (i <= 7) {
  sum += i;
  i += 2;
}
System.out.println(sum);
```

- The four additions give 16.
- The counter is 9 after the final update.
- There are five condition checks, including the last false check.

A bounded number-guessing loop

- The flag records success; the counter limits how long the loop may continue.
- Increment the attempt count once for each accepted integer guess.
- Use a fixed secret while tracing so the expected outcome is reproducible.

Source: supplied ISB campus slides. See speaker notes and ISB_Source_Map.md.

Worked problem: A bounded number-guessing loop

Task

Use secret 42 and at most five guesses. Test input 10, 80, 42 and report Low, High, then Correct.

Input: 10 80 42

Before running

Predict the output and identify the variables that change.

Complete ISB example (1/2)

```
public class IsbLecture10 {
  public static void main(String[] args) throws Exception {
    java.util.Scanner in = new java.util.Scanner(System.in);
    int secret = 42, attempts = 0;
    boolean done = false;
    while (attempts < 5 && !done) {
      int guess = in.nextInt();
      attempts++;
      if (guess == secret) {
        done = true;
```

Complete ISB example (2/2)

```
        System.out.println("Correct");
      } else {
        System.out.println(guess < secret ? "Low" : "High");
      }
    }
    System.out.println("Attempts: " + attempts);
  }

}
```

Worked trace and expected result

Guess	attempts	done / message
10	1	false / Low
80	2	false / High
42	3	true / Correct

Expected console output

```
Low
High
Correct
Attempts:  3
```

Extend the ISB example

Practice

Give an input sequence that exercises termination by the attempt limit rather than success.

Explain your reasoning

Five wrong integers, such as 1 2 3 4 5, produce five attempts with done still false.

Lecture summary

- Design terminating while loops
- counter, sentinel and flag control
- state across iterations

- Initialise sum and count to zero, then read the first mark.
- While the mark is not -1, accumulate it and read the next mark.
- If count is positive, divide using floating-point arithmetic. Otherwise print No data.

After-class practice

Use while to compute the digit sum of a nonnegative integer. Test 0, 7 and 204.

Syllabus reading
Deitel Ch 8 (as listed)
Next lecture: For and do-while loops